Hive Ransomware – Static Malware Analysis Report

Case study focused on triage, artifact extraction, and IOC identification

Tools: Bulk Extractor, file, sha256sum, md5sum, strings, UPX, Detect It Easy

Sample source: TheZoo repository

Method: Static Analysis Only

Author: Dário Alves | Role: Cybersecurity Student | Date: March 2026

Summary

Objective

This report presents a static analysis of a Hive ransomware sample obtained from the TheZoo repository.

The purpose of the analysis was to identify suspicious artifacts, extract relevant indicators of compromise (IOCs), and assess characteristics associated with ransomware behavior without executing the sample.

Scope and Methodology

The analysis was conducted in an isolated environment using a static triage approach only.

No live detonation or dynamic execution was performed.

The workflow included file identification, hash verification, artifact extraction, string inspection, and examination of suspicious content using tools such as Bulk Extractor, file, sha256sum, md5sum, strings, and Detect It Easy.

Key Findings

- The extracted files matched the MD5 and SHA256 hashes from TheZoo repository.
- The package contained both Windows and Linux artifacts.
- hive.bin_exe was identified as the main Windows executable.
- The executable was packed with UPX 3.96.
- After unpacking, the file structure became clearer for analysis.
- The sample contained ransomware related strings.
- RSA cryptography references were identified.
- Two .onion domains were found in the unpacked executable.

Sample Extraction and Hash Verification

After obtaining the Hive sample package from TheZoo repository, the archive contents were extracted in an isolated analysis environment.

Since the repository provides reference hashes for the extracted files, integrity verification was performed by comparing the calculated SHA256 and MD5 values of the extracted artifacts against the published reference values.

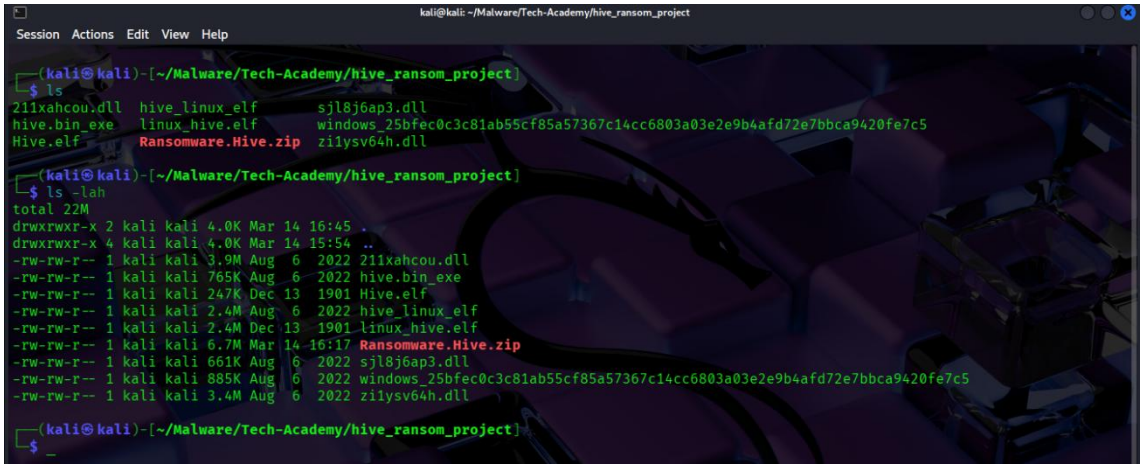

Figure 1. Extracted contents of the Hive sample package

The extracted package contained multiple artifacts associated with both Windows and Linux environments, including executable files, DLLs, and ELF binaries.

Hash Verification

Hash verification is a critical step in malware analysis, as it helps confirm the integrity and authenticity of the extracted files before any further examination is performed.

By comparing the calculated MD5 and SHA256 values with the reference hashes provided in the repository, it is possible to verify that the samples were not modified, corrupted, or altered during download and extraction.

In this case, all calculated MD5 and SHA256 hash values matched the corresponding reference values from the repository, confirming that the extracted files are intact and suitable for subsequent static analysis.

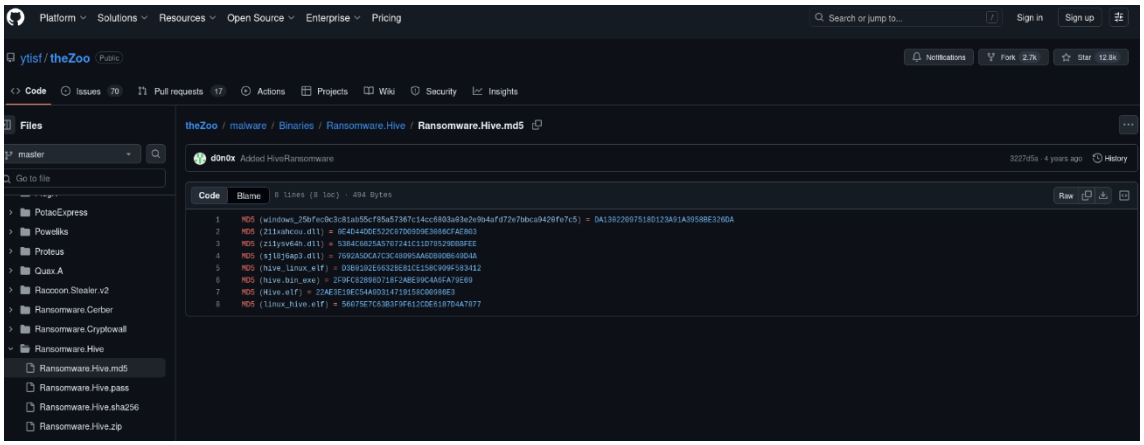

Figure 2. Reference MD5 hash values provided by the TheZoo repository for the extracted Hive sample files

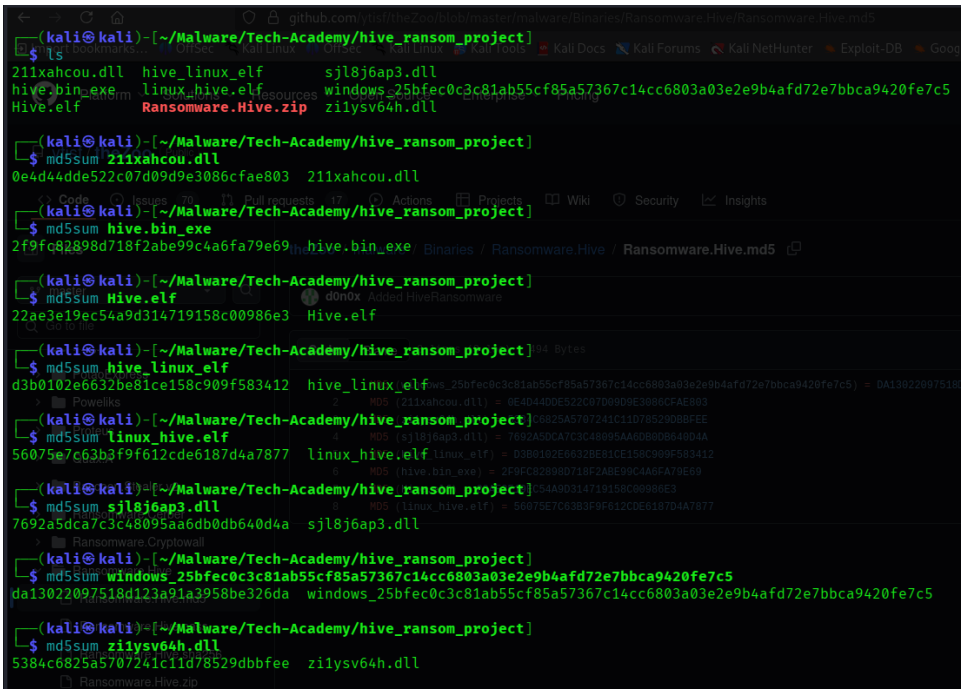

Figure 3. Locally calculated MD5 hash values for the extracted Hive sample files.

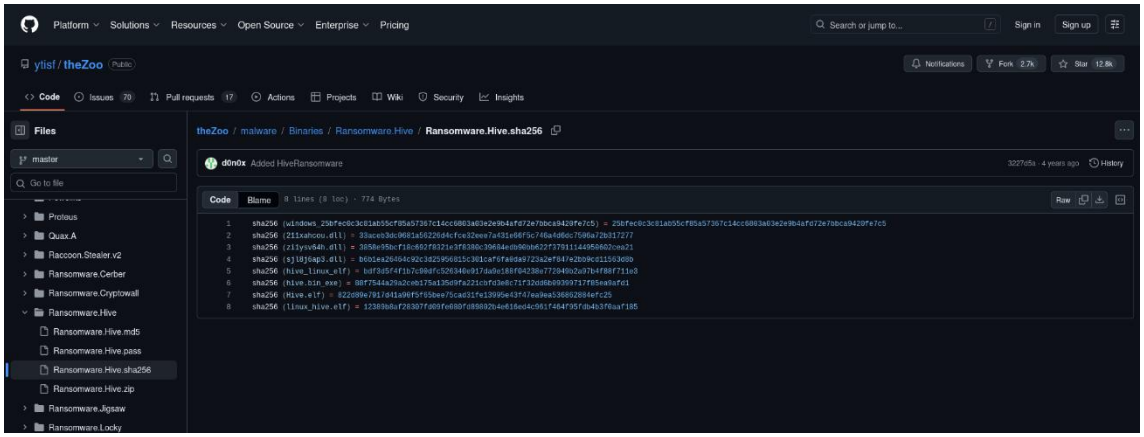

Figure 4. Reference SHA256 hash values provided by the TheZoo repository for the extracted Hive sample files

```
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ ls
211xahcou.dll  Hive.elf        linux_hive.elf       sjl8j6ap3.dll                                                    zi1ysv64h.dll
hive.bin_exe   hive_linux_elf  Ransomware.Hive.zip  windows_25bfec0c3c81ab55cf85a57367c14cc6803a03e2e9b4afd72e7bbca9420fe7c5

┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ sha256sum windows_25bfec0c3c81ab55cf85a57367c14cc6803a03e2e9b4afd72e7bbca9420fe7c5
25bfec0c3c81ab55cf85a57367c14cc6803a03e2e9b4afd72e7bbca9420fe7c5  windows_25bfec0c3c81ab55cf85a57367c14cc6803a03e2e9b4afd72e7bbca9420fe7c5

┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ sha256sum 211xahcou.dll
33aceb3dc0681a56226d4cfce32eee7a431e66f5c746a4d6dc7506a72b317277  211xahcou.dll
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ sha256sum zi1ysv64h.dll
3858e95bcf18c692f8321e3f8380c39684edb90bb622f37911144950602cea21  zi1ysv64h.dll

┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ sha256sum sjl8j6ap3.dll
b6b1ea26464c92c3d25956815c301caf6fa0da9723a2ef847e2bb9cd11563d8b  sjl8j6ap3.dll

┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ sha256sum hive_linux_elf
bdf3d5f4f1b7c90dfc526340e917da9e188f04238e772049b2a97b4f88f711e3  hive_linux_elf
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ sha256sum hive.bin_exe
88f7544a29a2ceb175a135d9fa221cbfd3e8c71f32dd6b09399717f85ea9afd1  hive.bin_exe

┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ sha256sum Hive.elf
822d89e7917d41a90f5f65bee75cad31fe13995e43f47ea9ea536862884efc25  Hive.elf
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ sha256sum linux_hive.elf
12389b8af28307fd09fe080fd89802b4e616ed4c961f464f95fdb4b3f0aaf185  linux_hive.elf

┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
```

Figure 5. Locally calculated SHA256 hash values for the extracted Hive sample files.

File Type Identification

Following integrity verification, an initial file type identification step was performed using the file command.

This stage is important in malware analysis because it helps determine the nature of each extracted artifact, including whether the files are Windows executables, dynamic link libraries, Linux ELF binaries, or compressed archives.
This preliminary classification supports the selection of the most relevant samples for deeper static analysis.

```
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ file *
211xahcou.dll:                                          PE32+ executable for MS Windows 6.01 (GUI), x86-64 (stripped to external PDB), 6 sections
hive.bin_exe:                                           PE32 executable for MS Windows 6.01 (GUI), Intel i386 (stripped to external PDB), UPX compressed, 3 sections
Hive.elf:                                               ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.
2, BuildID[sha1]=9a3806ab139e3a10231daa5b76db84a3684791e7, for GNU/Linux 3.2.0, stripped
hive_linux_elf:                                         ELF 64-bit LSB executable, x86-64, version 1 (FreeBSD), statically linked, stripped
linux_hive.elf:                                         ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, stripped
Ransomware.Hive.zip:                                    Zip archive data, made by v3.1, extract using at least v2.0, last modified Aug 06 2022 14:25:50, uncompressed size
782394, method=AES Encrypted
sjl8j6ap3.dll:                                          PE32+ executable for MS Windows 5.02 (GUI), x86-64 (stripped to external PDB), 10 sections
windows_25bfec0c3c81ab55cf85a57367c14cc6803a03e2e9b4afd72e7bbca9420fe7c5: PE32 executable for MS Windows 6.01 (console), Intel i386 (stripped to external PDB), UPX compressed, 3 sections
zi1ysv64h.dll:                                          PE32+ executable for MS Windows 6.01 (console), x86-64 (stripped to external PDB), 6 sections
```

Figure 6. File type identification of the extracted Hive sample artifacts using the file command.

The output shows that the extracted package contains a mixture of Windows and Linux artifacts.

Several files were identified as PE32 or PE32+ executables for Microsoft Windows, including DLL files and console or GUI executables.

In addition, multiple ELF 64-bit binaries were identified, indicating Linux targeted components within the same sample package.

This confirms that the Hive package includes cross platform artifacts and not a single standalone executable.

Among the identified files, hive.bin_exe stands out as a relevant Windows executable for further analysis, as it was identified as a PE32 file for MS Windows and appears to be UPX compressed.

This makes it a strong candidate for deeper static inspection.

Static Analysis of hive.bin_exe

A heuristic scan of hive.bin_exe was performed using Detect It Easy Command Line (diec --heuristicscan) in order to obtain additional structural indicators beyond the basic file identification stage.

The output confirmed multiple characteristics consistent with UPX packing, including packer like entry point behavior, UPX import patterns, section characteristics associated with UPX, and compressed or packed data.

These findings further support the conclusion that the executable is packed, which may reduce the visibility of its original structure and embedded strings during initial static analysis.

Figure 7. Heuristic scan results of hive.bin_exe using Detect It Easy

The string listing did not produce meaningful results, likely due to the presence of UPX packing, which can reduce the visibility of useful embedded strings during static analysis.

Analysis of the Unpacked hive.bin_exe

```
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ ls
211xahcou.dll  hive.bin_exe.copy  hive_linux_elf  Ransomware.Hive.zip  windows_25bfec0c3c81ab55cf85a57367c14cc6803a03e2e9b4afd72e7bbca9420fe7c5
hive.bin_exe   Hive.elf           linux_hive.elf  sjl8j6ap3.dll        zi1ysv64h.dll

┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ upx -d -o hive.bin_exe.unpacked hive.bin_exe.copy
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2024
UPX 4.2.4       Markus Oberhumer, Laszlo Molnar & John Reiser    May 9th 2024

        File size         Ratio      Format      Name
   --------------   ------ -----   -----------   -----------
   2292794 <-     782394   34.12%    win32/pe     hive.bin_exe.unpacked

Unpacked 1 file.
```

Figure 8. sample unpacking with UPX

```
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ file hive.bin_exe.unpacked

hive.bin_exe.unpacked: PE32 executable for MS Windows 6.01 (GUI), Intel i386 (stripped to external PDB), 6 sections

┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ diec --heuristicscan hive.bin_exe.unpacked

[!] To get the full heuristic scan result use '--verbose'
[HEUR/About] Generic Heuristic Analysis by DosX (@DosX_dev)
[HEUR] Scanning has begun!
[HEUR] Scan completed.
PE32
    Compiler: Go($ B$ R$ l$ •$ •$ •$ •$ •$ •$ •$ •$
                                                $ 8$)
```

Figure 9. File type and heuristic inspection of the sample

After unpacking the main Windows executable with UPX, a second round of static inspection was performed on the resulting artifact in order to improve visibility into its internal structure and embedded content.

The unpacked file was identified as a PE32 Windows executable with six sections, allowing a clearer view of the sample compared to the original packed version.

Additional analysis suggested Go compilation indicators, while string extraction revealed ransom content, including instructions for victims, warnings against contacting authorities, and recommendations not to use third party decryption tools.

These findings strongly support the classification of the sample as ransomware.

Additionally, the analysis indicated the use of the RSA algorithm in the key cryptography component, which is consistent with ransomware families that rely on asymmetric encryption mechanisms to protect or manage decryption keys.

```
rned zero, running on unsupported hardwarereflect.Value.Interface: cannot return value obtained from unexported field or methodx509: failed to parse public key (use ParsePKIXPublicKey inste
ad for this key formatjQueryPerformanceFrequency overflow 32 bit divider, check nosplit discussion to proceedasn1: time did not serialize back to the original value and may be invalid: give
n %q, but serialized as %q@w
To decrypt all the data you will need to purchase our decryption software.
 - Do not report to authorities. The negotiation process will be terminated immediately and the key will be erased.
 - Do not try to decrypt data using third party software. It may cause irreversible damage.
 - Do not fool yourself. Encryption has perfect secrecy and it's impossible to decrypt without knowing the key.
```

Figure 10. Ransom Note

Artifact Extraction with Bulk Extractor

kali | Malware | Tech-Academy | hive_ransom_project | report

Name	Size	Type	Date Modified
report.xml	12.5 KiB	XML document	Today
url.txt	485 bytes	Plain text document	Today
domain.txt	326 bytes	Plain text document	Today
url_histogram.txt	307 bytes	Plain text document	Today
url_services.txt	229 bytes	Plain text document	Today
domain_histogram.txt	229 bytes	Plain text document	Today
zip.txt	0 bytes	Empty document	Today
winprefetch.txt	0 bytes	Empty document	Today
winpe_carved.txt	0 bytes	Empty document	Today
winpe.txt	0 bytes	Empty document	Today
winlnk.txt	0 bytes	Empty document	Today
windirs.txt	0 bytes	Empty document	Today
vcard.txt	0 bytes	Empty document	Today
utmp_carved.txt	0 bytes	Empty document	Today
url_searches.txt	0 bytes	Empty document	Today
url_microsoft-live.txt	0 bytes	Empty document	Today
url_facebook-id.txt	0 bytes	Empty document	Today
url_facebook-address.txt	0 bytes	Empty document	Today
unrar_carved.txt	0 bytes	Empty document	Today
telephone_histogram.txt	0 bytes	Empty document	Today
telephone.txt	0 bytes	Empty document	Today
tcp_histogram.txt	0 bytes	Empty document	Today
tcp.txt	0 bytes	Empty document	Today
sqlite_carved.txt	0 bytes	Empty document	Today
sin.txt	0 bytes	Empty document	Today

Figure 11. Output files generated by Bulk Extractor

```
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project/report]
└─$ cat domain_histogram.txt
# BANNER FILE NOT PROVIDED (-b option)
# BULK_EXTRACTOR-Version: 2.1.1
# Feature-Recorder: domain
# Filename: hive.bin_exe.unpacked
# Histogram-File-Version: 1.1
n=1     hivecust6vhekztbqgdnkks64ucehqacge3dij3gyrrpdp57zoq3ooqd.onion
```

Figure 12. Domain related artifacts detected .onion domain

```
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project/report]
└─$ cat url_services.txt
# BANNER FILE NOT PROVIDED (-b option)
# BULK_EXTRACTOR-Version: 2.1.1
# Feature-Recorder: url
# Filename: hive.bin_exe.unpacked
# Histogram-File-Version: 1.1
n=1     ://hivecust6vhekztbqgdnkks64ucehqacge3dij3gyrrpdp57zoq3ooqd.onion
```

Figure 13. URL related artifacts detected .onion domain

```
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project/report]
└─$ cat domain.txt
# BANNER FILE NOT PROVIDED (-b option)
# BULK_EXTRACTOR-Version: 2.1.1
# Feature-Recorder: domain
# Filename: hive.bin_exe.unpacked
# Feature-File-Version: 1.1
1309072 hivecust6vhekztbqgdnkks64ucehqacge3dij3gyrrpdp57zoq3ooqd.onion  sful readhttp://hivecust6vhekztbqgdnkks64ucehqacge3dij3gyrrpdp57zoq3ooqd.onion/http://hiveleak
```

Figure 14. Domain related artifacts detected reference to hiveleak

After unpacking the main executable, Bulk Extractor identified network artifacts, including .onion references.

The presence of Tor hidden service addresses is a relevant indicator in the context of ransomware analysis, as it may suggest hidden communication, victim access infrastructure, or extortion resources.

One extracted reference also includes the term hiveleak, which may indicate a leak site string within the sample.

```
┌──(kali㉿kali)-[~/Malware/Tech-Academy/hive_ransom_project]
└─$ strings -a hive.bin_exe.unpacked | grep -aoE '[a-z0-9]{16,56}\.onion' | sort -u

hivecust6vhekztbqgdnkks64ucehqacge3dij3gyrrpdp57zoq3ooqd.onion
hiveleakdbtnp76ulyhi52eag6c6tyc3xw7ez7iqy6wc34gd2nekazyd.onion
```

Figure 15. Extracted .onion references

String extraction from the unpacked sample revealed two .onion references embedded in the executable.

These findings suggest the presence of Tor accessible infrastructure within the sample, including a possible victim portal and a leak resource.

The presence of these hidden service references strengthens the ransomware assessment and indicates network oriented artifacts commonly associated with extortion operations.

Conclusion

This report presented a structured static analysis of a Hive ransomware sample obtained from TheZoo repository.

The analysis confirmed that the extracted files matched the reference MD5 and SHA256 hashes, supporting the integrity of the artifacts examined.

Initial triage identified a package containing both Windows and Linux components, with hive.bin_exe selected as the main Windows artifact for deeper inspection.

Static analysis showed that the executable was packed with UPX and later successfully unpacked, allowing improved visibility into its internal structure.

Further inspection revealed ransomware characteristics, including victim oriented content, cryptographic references, and embedded .onion addresses associated with Tor hidden services.

Overall, the findings are consistent with ransomware behavior and demonstrate how static analysis can be used to identify relevant technical indicators without executing the sample.

This report is limited to static analysis only, and therefore runtime behavior, persistence mechanisms, and live network communication could not be confirmed.